

RAPPORT DE PROJET : TRAITEMENT DE DONNÉES DE PREMIÈRE ANNÉE

Élèves 1A

À REMPLIR

Étudiants :

Groupe 25

Ilan GUEDJ

Jules FRULLI

Thomas ETIENNE

Alexandre FRATTER-BARDY

Responsable :

Johann FAOUZI

Encadrant :

Clément VALOT

Table des matières

Introduction	1
1 Modélisation et Diagrammes UML	2
1.1 Diagramme d'état	2
1.2 Diagramme de classe	3
1.3 Diagramme d'action	4
2 Architecture et choix techniques	5
2.1 Les outils utilisés	5
2.2 La programmation orientée objet	5
2.3 Les choix d'utilisations	6
3 Implémentation de la programmation orientée objet (POO)	7
3.1 Les classes modèles	7
3.2 Les classes Loaders	7
3.3 Les classes services	8
3.4 Les avantages de cette architecture logicielle	8
4 Analyse des bases de données et traitements	10
4.1 Présentation globale des données	10
4.2 Analyse détaillée d'un jeu de données : le cas du Basketball	10
4.2.1 Structure et types de variables	10
4.2.2 Nettoyage et gestion des valeurs manquantes	10
4.2.3 Transformation spécifique : conversion des mesures impériales	10
4.3 L'harmonisation	11
5 Assurance qualité du code	12
5.1 Outils qualitatifs	12
5.2 Mise en place des tests	12
6 Présentation des résultats	14
6.1 L'interface utilisateur	14
6.2 Consultation des statistiques	14
6.3 Les comparateurs graphiques	15
6.4 Analyses physiologiques et data visualisation	16
6.5 Édition et mise à jour dynamique	16
7 Conclusion	18
8 Annexes	19
8.1 Avis personnels	19
8.1.1 Jules	19
8.1.2 Ilan	19
8.1.3 Thomas	19
8.1.4 Alexandre	19

Introduction

Suite au succès des Jeux Olympiques et Paralympiques de Paris 2024, la gestion et l'analyse des données sportives sont devenues des enjeux stratégiques majeurs. C'est dans ce contexte stimulant que le Ministère des Sports, de la Jeunesse et de la Vie Associative souhaite développer une nouvelle application dédiée à la gestion centralisée des résultats de compétitions sportives. Cependant, l'histoire récente des grands projets informatiques de l'État est marquée par des échecs très coûteux (tels que les logiciels Scribe ou les systèmes de paie de l'Éducation nationale). Par conséquent, les attentes en matière de fiabilité, de conception et de résultats pour cette nouvelle application sont exceptionnellement élevées.

L'objectif principal est de concevoir et de développer une application robuste capable de modéliser l'écosystème sportif à travers la création d'entités telles que des matchs, des compétitions, des équipes ou encore des joueurs. L'application doit non seulement sauvegarder les caractéristiques pertinentes de ces acteurs, mais aussi offrir la possibilité d'intégrer dynamiquement les résultats de nouvelles compétitions afin de mettre à jour leurs statistiques de manière automatisée. Une des complexités majeures de ce cahier des charges réside dans l'exigence de modularité : l'application doit pouvoir fonctionner sur plusieurs jeux de données différents. Si certaines caractéristiques demeurent spécifiques à une discipline, le socle central des fonctionnalités se doit d'être universel et opérationnel indépendamment du sport traité.

Pour ce faire, nous devons manipuler divers jeux de données bruts. Tout l'enjeu a été de concevoir un système capable d'ingérer, de nettoyer et de structurer ces informations afin de les rendre exploitables par notre application, quel que soit le contexte sportif.

Afin de répondre efficacement à ces enjeux de modélisation et de flexibilité, nous avons raisonné en programmation orientée objet. Cette approche s'est révélée particulièrement pertinente, car elle permet de traduire fidèlement les concepts du monde réel en objets informatiques interagissant entre eux. Enfin, pour répondre à la contrainte stricte de maintenabilité imposée par le Ministère, une attention toute particulière a été accordée à l'architecture logicielle. L'application a été codée dans le strict respect des bonnes pratiques de programmation, garantissant ainsi un code propre, documenté et évolutif, prêt à être repris et enrichi par de futures équipes de développeurs sans risquer l'impasse technique.

1 Modélisation et Diagrammes UML

1.1 Diagramme d'état

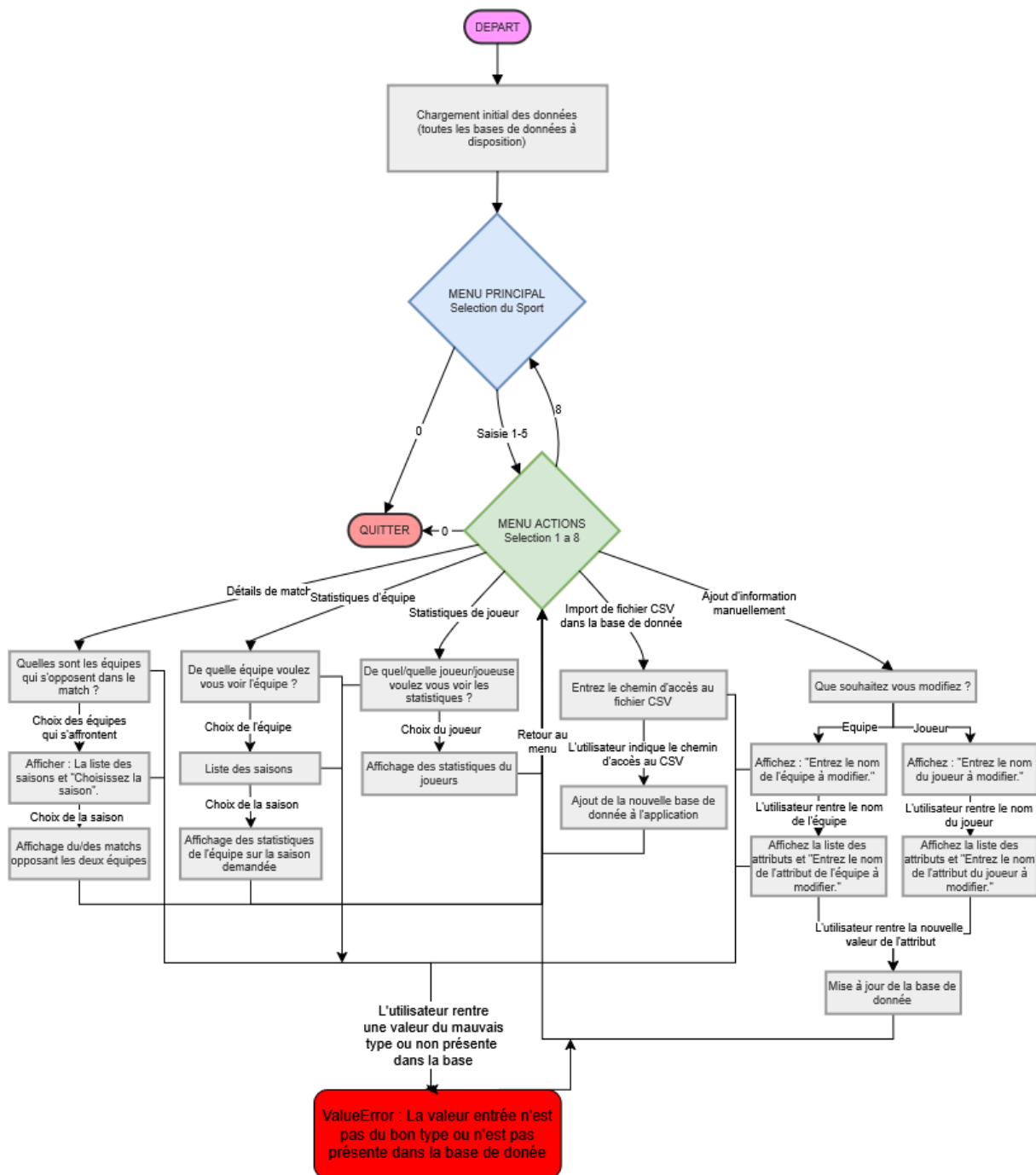


FIGURE 1.1 : Diagramme d'état de l'application

Pour éviter de surcharger le diagramme, les fonctionnalités qui permettent de produire des graphiques n'ont pas été représentées dans le diagramme d'état.

1.2 Diagramme de classe

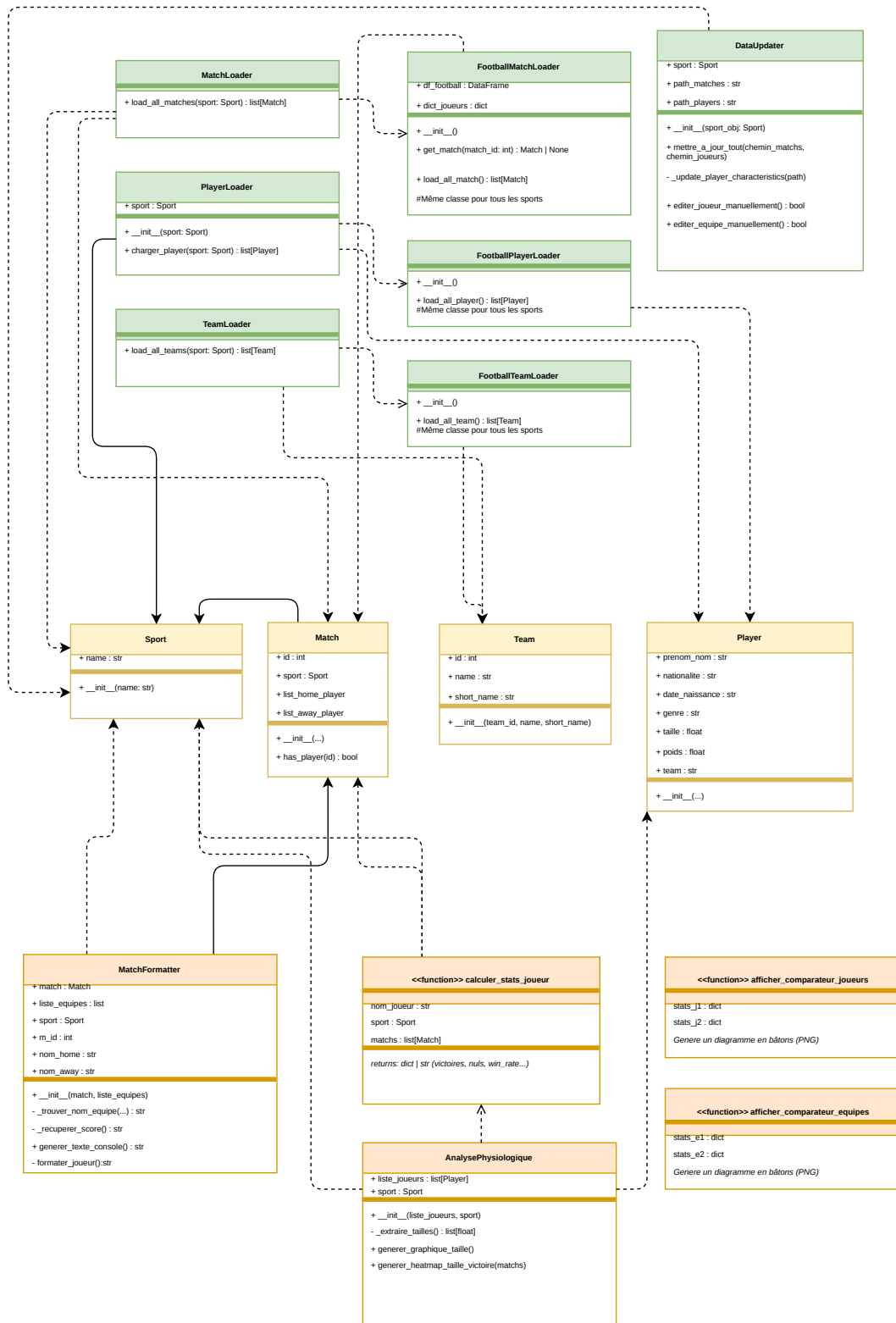


FIGURE 1.2 : Diagramme de classe de l'application

Le choix a été fait de ne pas mettre tous les loaders spécifiques des sports (seulement celui du foot, qui est le même que les autres) pour éviter de surcharger le diagramme.

1.3 Diagramme d'action

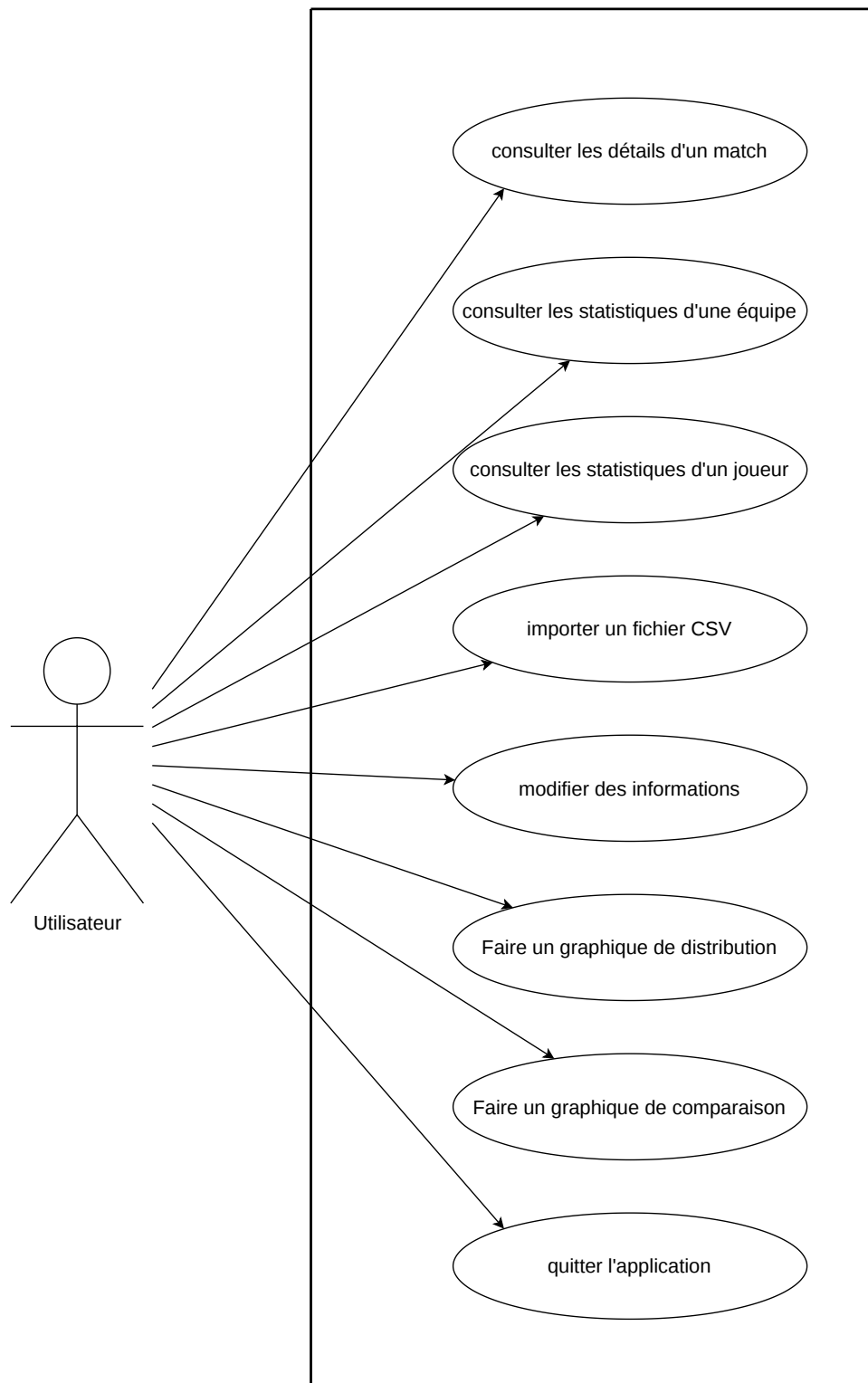


FIGURE 1.3 : Diagramme d'action de l'application

2 Architecture et choix techniques

2.1 Les outils utilisés

Le langage de programmation utilisé lors de ce projet est *Python 3*. La bibliothèque *Pandas* est au cœur du traitement des fichiers CSV. Elle nous permet de réaliser toutes les tâches dont nous avons besoin pour répondre au cahier des charges : lecture, filtrage et sauvegarde des données. De plus, nous avons utilisé la bibliothèque *NumPy* pour optimiser les calculs des statistiques sur de grandes séries de données, par exemple pour le calcul de moyennes ou d'indicateurs clés sur les joueurs. Concernant la visualisation des données, la bibliothèque *Matplotlib* a été utilisée pour sa capacité à générer des graphiques, qu'ils soient complexes ou non, comme des histogrammes ou des heatmaps. Enfin, l'utilisation des modules natifs *os* et *sys* permet de garantir la portabilité du code : gestion des chemins relatifs des fichiers de données et ce, peu importe l'ordinateur sur lequel le script est exécuté.

2.2 La programmation orientée objet

Lors de la réalisation de ce projet, nous avons fait face à un problème de données. En effet, ces données font référence à des sports différents. Elles sont donc, par nature, hétérogènes. Nous avons par exemple des données sur le football et d'autres sur le tennis. Mais ces dernières sont séparées selon le genre de l'athlète. Par ailleurs, les fichiers CSV n'ont ni les mêmes colonnes, ni les mêmes unités de mesure d'un sport à l'autre. Ainsi, une approche avec de simples listes ou dictionnaires aurait créé un code peu lisible, rempli de conditions sur le sport choisi. Grâce à la programmation orientée objet (POO), nous avons créé une manière universelle de représenter nos sports, matchs et athlètes. Peu importe comment le joueur est décrit dans le fichier CSV, les loaders extraient la donnée et fabriquent un objet *Player* avec des attributs universels comme, par exemple, la taille ou le nom. Aussi, les responsabilités sont séparées : le fichier *main.py* ne fait que gérer le menu, les loaders ne font que traduire les CSV en objets et les différents services proposés ne font que générer des graphiques ou des statistiques à partir de ces objets.

```
try:
    if "-" in h_str or "'" in h_str:
        parts = h_str.replace("'", "-").split("-")
        if len(parts) == 2:
            pieds = float(parts[0])
            pouces = float(parts[1])
            h_cm = (pieds * 30.48) + (pouces * 2.54)
            if h_cm > 140:
                tailles.append(h_cm)
```

FIGURE 2.1 : Extracteur de taille

Cela représente un extrait de notre code portant sur le nettoyage des données. Nous suivons une logique de standardisation de la taille de l'objet *Player*, nous convertissons ainsi les données impériales (pieds-pouces) en système métrique (cm).

2.3 Les choix d'utilisations

Nous avons fait le choix d'utiliser Pandas et non le module CSV natif. En effet, Pandas permet de gérer facilement les données textuelles incomplètes. Pandas permet également de filtrer les données de manière plus efficace que si nous n'avions pas utilisé cette bibliothèque. Par exemple, dans notre classe *DataUpdater*, pour trouver un joueur quand on ne sait pas si la colonne s'appelle *first_name* ou *player_name*, Pandas fusionne virtuellement les colonnes des noms et applique un filtre de recherche.

```
# On fusionne toutes les colonnes de noms trouvées
noms_complets_virtuels = df[colonnes_noms].astype(str).agg(' '.join, axis=1)

masque = noms_complets_virtuels.str.contains(nom_recherche, case=False, na=False)
resultats = df[masque]
```

FIGURE 2.2 : Extrait du filtre de recherche

Pour répondre au cahier des charges, nous avons également utilisé la bibliothèque Matplotlib qui nous permet de réaliser des graphiques complets comme, par exemple, l'analyse de la taille des joueurs avec le pourcentage de victoires. Pour représenter la corrélation entre ces deux variables, nous avons utilisé la fonction *plt.hexbin()*. En effet, quand on croise deux variables comme la taille et le pourcentage de victoires pour des milliers d'individus, un graphique classique crée un nuage de points illisible. Cette fonction regroupe donc les joueurs par alvéoles et utilise la couleur pour montrer la concentration des individus dans une certaine zone. Cela offre une lecture plus fluide.

```
# Carte de chaleur avec un dégradé de Jaune (peu de joueurs) à Rouge (beaucoup de joueurs)
heatmap = plt.hexbin(tailles_valides, win_rates_valides, gridsize=15, cmap='YlOrRd', mincnt=1)
```

FIGURE 2.3 : Configuration d'une carte de chaleur avec Matplotlib

La mise à jour des données est isolée dans la classe *DataUpdater* car, dans le cadre d'un outil utilisé par plusieurs acteurs, modifier les données en mémoire vive ne sert à rien si ce n'est pas sauvegardé. Ainsi, cette classe garantit que toute modification des données ou tout ajout de nouvelles données est réécrit de manière sécurisée et que les Loaders sont rechargés par la suite pour que l'application soit toujours à jour.

3 Implémentation de la programmation orientée objet (POO)

Afin de gérer la diversité de nos données sportives et d'éviter un code difficile à maintenir, notre application repose sur une architecture logicielle modulaire. Nous avons divisé notre code en trois grandes familles de classes, respectant ainsi le principe de responsabilité unique : les modèles pour encapsuler la donnée, les loaders pour extraire et nettoyer, et les services pour traiter et analyser.

3.1 Les classes modèles

Nous avons créé des classes spécifiques comme *Player*, *Team* et *Sport* qui agissent comme des conteneurs de données. Au lieu de manipuler directement des lignes de DataFrames ou des dictionnaires tout au long du code, chaque élément pertinent de nos fichiers CSV est transformé en un objet. L'objectif principal de cette démarche est la standardisation et la sécurité des données.

Par exemple, qu'un athlète provienne du fichier *atp_players_2024.csv* (tennis) ou *player.csv* (football), il devient un objet *Player* avec des attributs stricts et communs comme *prenom_nom*, *taille* ou *genre*. Cette encapsulation garantit que le reste du programme n'a pas à se soucier des noms de colonnes originaux du CSV. Si une clé de dictionnaire vient à changer dans les données brutes, l'erreur ne se propagera pas dans toute l'application : la donnée est verrouillée et uniformisée au sein de l'objet. Cela permet à l'application de traiter n'importe quel sport de manière universelle.

```
for i in range(len(df_basketball)):
    prenom = df_basketball.loc[i, "first_name"]
    nom = df_basketball.loc[i, "last_name"]
    nom_complet = f"{prenom} {nom}"
    joueur = Player(None, None, None, None, None, None, None)

    joueur.prenom_nom = nom_complet
    joueur.nationalite = None
    joueur.date_naissance = df_basketball.loc[i, "birthdate"]
    joueur.genre = None
    joueur.height = df_basketball.loc[i, "height"]
    joueur.poids = df_basketball.loc[i, "weight"]
    joueur.team = df_basketball.loc[i, "team_id"]
```

FIGURE 3.1 : Création d'un joueur et assignation des attributs

3.2 Les classes Loaders

Pour protéger la logique de notre application des incohérences souvent présentes dans les jeux de données bruts, nous avons isolé la lecture des fichiers. La mission des classes *MatchLoader*, *PlayerLoader* et *TeamLoader* est d'ouvrir les fichiers CSV, de gérer les valeurs manquantes via des blocs *try/except*, de convertir les types de variables (par exemple, transformer une chaîne de caractères en entier) et de fabriquer les objets finaux.

L'intérêt de cette approche est que chaque sport possède son propre comportement lors du chargement, sans impacter le programme principal. Par exemple, le loader du tennis gère

la séparation des données en deux fichiers distincts (ATP pour les hommes et WTA pour les femmes). Le loader va lire ces deux fichiers séparément, leur ajouter une colonne *genre* pour garder la trace de la catégorie, puis utiliser *Pandas* pour les concaténer. Il renvoie ainsi une liste unique et complète de joueurs de tennis prête à être analysée.

```
df_atp["genre"] = "Homme"
df_wta["genre"] = "Femme"

df_tennis = pd.concat([df_atp, df_wta], ignore_index=True)
```

FIGURE 3.2 : Concaténation de deux dataframes dans TennisPlayerLoader

3.3 Les classes services

Les services sont les classes qui effectuent les calculs complexes ou génèrent les affichages, sans stocker la donnée de manière permanente. Nous avons par exemple la classe *AnalysePhysiologique*, qui reçoit la liste des objets *Player* et s'occupe de toute la visualisation (génération des histogrammes et des heatmaps). Le fait de l'isoler dans une classe évite de surcharger le fichier principal **main.py** avec de multiples paramétrages Matplotlib et permet de réutiliser ces graphiques facilement.

De la même manière, d'autres classes de services gèrent la logique métier de l'application. La classe *ChampionshipPointsCalculator* s'occupe du calcul des points de victoires ou de défaites. Puisque les règles de calcul diffèrent selon les sports, cette classe centralise cette complexité : le programme principal se contente d'appeler une méthode unique, et le service applique la bonne formule. Enfin, la classe *DataUpdater* gère l'aspect persistance des données. Elle isole les opérations d'écriture afin de s'assurer que toute modification manuelle ou importation de nouvelle compétition n'écrase pas accidentellement nos historiques.

```
class AnalysePhysiologique:
    """
    Classe responsable de l'analyse et de la visualisation
    des données morphologiques des joueurs.
    """
    def __init__(self, liste_joueurs, sport):
        self.liste_joueurs = liste_joueurs
        self.sport = sport
```

FIGURE 3.3 : Classe AnalysePhysiologique

3.4 Les avantages de cette architecture logicielle

Cette architecture nous a demandé un effort de modélisation et de conception au départ, mais elle apporte des bénéfices majeurs pour le projet.

Premièrement, la maintenabilité : le fichier principal *main.py* se contente d'afficher les menus et d'appeler des méthodes aux noms explicites, agissant uniquement comme un chef d'orchestre. En cas de bug lors d'un calcul, nous savons exactement dans quelle classe de service chercher, sans avoir à parcourir les lignes de code d'interface.

Deuxièmement, l'application est évolutive. Si nous souhaitons ajouter un nouveau sport possédant ses propres règles d'importation, comme le rugby, il n'est pas nécessaire de modifier

le système de menus, les algorithmes de calcul ou les générateurs de graphiques. Il suffira d'ajouter les fichiers CSV correspondants et de créer une simple classe *RugbyLoader*. Le reste de l'application s'adaptera automatiquement pour traiter ces nouvelles entités.

4 Analyse des bases de données et traitements

4.1 Présentation globale des données

Notre application traite des données issues de cinq sports différents : le football, le tennis, le basketball, le volleyball et League of Legends. D’autres bases de données concernant d’autres sports ont été fournies mais nous avons fait le choix de ne travailler qu’à partir de ces cinq premiers sports. Les données sont stockées au format CSV. Étant donné que ces jeux de données proviennent de sources distinctes, ils présentent une forte hétérogénéité. Les noms des colonnes diffèrent selon les sports (par exemple *first_name* et *last_name* pour le basket, contre *name_first* et *name_last* pour le tennis), tout comme les unités de mesure utilisées. Pour illustrer notre méthode de traitement, nous détaillerons ici l’analyse et le nettoyage du jeu de données du Basketball.

4.2 Analyse détaillée d’un jeu de données : le cas du Basketball

4.2.1 Structure et types de variables

Le fichier *player.csv* du Basketball recense les athlètes professionnels. Chaque ligne représente un joueur unique. Parmi les variables brutes à notre disposition, nous exploitons principalement les variables textuelles telles que *first_name*, *last_name*, *team_name* et les variables numériques stockées sous format texte telles que *height* (taille) et *weight* (poids).

```
person_id,first_name,last_name,birthdate,height,weight,jersey,position,team_id
1630173,Precious,Achiuwa,1999-09-19,6-8,225,5,Forward,1610612761
203500,Steven,Adams,1993-07-2,6-11,265,4,Center,1610612763
1628389,Bam,Adebayo,1997-07-18,6-9,255,13,Center-Forward,1610612748
1630534,Ochai,Agbaji,2000-04-2,6-5,215,30,Guard,1610612762
```

FIGURE 4.1 : Extrait du *player.csv* de basketball

4.2.2 Nettoyage et gestion des valeurs manquantes

Un jeu de données brut contient souvent des valeurs aberrantes ou manquantes. Lors de la phase de chargement effectuée par notre classe *BasketballPlayerLoader*, nous appliquons un nettoyage systématique. Nous utilisons les fonctionnalités de la bibliothèque Pandas pour filtrer ces données (par exemple via *.dropna()*) ou nous les gérons au niveau de l’extraction par des blocs *try/except*. Si un joueur ne possède ni taille ni poids valides dans le fichier source, une valeur par défaut de 0 ou *None* lui est attribuée temporairement, afin de ne pas faire échouer les calculs statistiques globaux ou de ne pas l’exclure purement et simplement de l’affichage.

4.2.3 Transformation spécifique : conversion des mesures impériales

La particularité majeure de ce jeu de données est son système de mesure américain. La taille y est renseignée en pieds et pouces sous forme de chaîne de caractères (ex. : “6-8” pour 6 pieds et 8 pouces). Pour que ces données puissent être comparées avec celles du football ou du volleyball, il fallait les convertir dans le système métrique.

Nous avons donc implémenté un algorithme d’extraction et de conversion. Pour la taille, le code divise la chaîne de caractères grâce à la méthode *.split(“-”)*. Il multiplie ensuite la première

valeur représentant les pieds par 30,48 et la seconde représentant les pouces par 2,54, avant d'additionner les deux pour obtenir la taille exacte en centimètres.

4.3 L'harmonisation

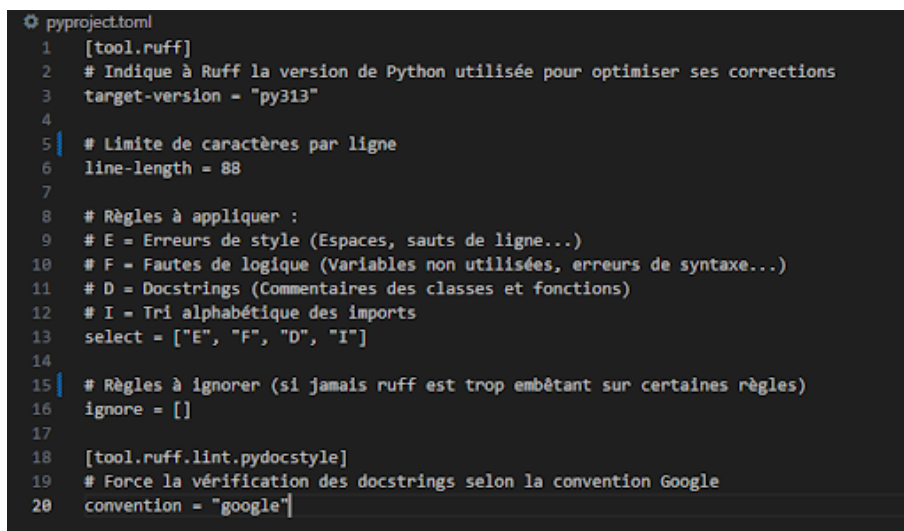
Ce travail de nettoyage et de transformation a été adapté à la spécificité de chaque jeu de données. À titre d'exemple, pour la base de données du Tennis, les données étaient scindées en deux fichiers distincts (ATP pour les hommes, WTA pour les femmes). Le traitement de ce jeu de données a nécessité l'ajout d'une colonne de genre et la concaténation des deux fichiers via Pandas avant de procéder à l'extraction. Ces traitements spécifiques garantissent que, in fine, toutes les données convergent vers un modèle unique et exploitable par notre application.

5 Assurance qualité du code

5.1 Outils qualitatifs

Pour garantir un code propre et fiable tout au long du projet, nous avons mis en place plusieurs outils qualitatifs. Tout d'abord, nous avons choisi d'utiliser Ruff, qui a l'avantage de combiner deux fonctions essentielles : un linter et un formateur. L'utilité du linter est d'analyser le code en profondeur pour prévenir les bugs potentiels, détecter les erreurs de logique et vérifier le respect des bonnes pratiques de programmation. En complément, le formateur agit comme un metteur en page automatique dont le but est d'harmoniser l'esthétique du fichier (par exemple en gérant les espaces ou la longueur des lignes) afin d'offrir une présentation visuelle standardisée. L'utilisation conjointe de ces deux fonctionnalités au sein de Ruff nous a ainsi permis de maintenir une syntaxe claire et d'appliquer naturellement la convention Google pour documenter nos classes.

Afin de pouvoir correctement standardiser notre code, nous avons créé un fichier *pyproject.toml* qui recense les règles à appliquer par notre linter. Pour cela, nous avons en premier lieu renseigné la version de Python utilisée : déjà définie dans notre fichier *README.md*. Nous avons ensuite choisi de définir la longueur maximale de nos lignes à 88, taille standard pour le code. Un ensemble de règles à appliquer par le linter a également été défini sur la base d'une couverture maximale des bases de la lisibilité du code : les erreurs de style, les fautes de logique, les docstrings et le tri alphabétique des imports. Enfin, la convention Google a été paramétrée dans ce fichier afin que la vérification des docstrings se fasse bien selon ce critère.



```
1 [tool.ruff]
2 # Indique à Ruff la version de Python utilisée pour optimiser ses corrections
3 target-version = "py313"
4
5 # Limite de caractères par ligne
6 line-length = 88
7
8 # Règles à appliquer :
9 # E = Erreurs de style (Espaces, sauts de ligne...)
10 # F = Fautes de logique (Variables non utilisées, erreurs de syntaxe...)
11 # D = Docstrings (Commentaires des classes et fonctions)
12 # I = Tri alphabétique des imports
13 select = ["E", "F", "D", "I"]
14
15 # Règles à ignorer (si jamais ruff est trop embêtant sur certaines règles)
16 ignore = []
17
18 [tool.ruff.lint.pydocstyle]
19 # Force la vérification des docstrings selon la convention Google
20 convention = "google"
```

FIGURE 5.1 : Paramétrage de l'outil d'analyse statique Ruff pour l'assurance qualité du projet

Ensuite, pour évaluer objectivement la qualité de notre travail, nous nous sommes appuyés sur deux métriques quantitatives. D'une part, nous avons utilisé l'évaluation du linter comme un repère. Avec Ruff, notre but était d'atteindre zéro erreur. Par conséquent, en corrigeant systématiquement chaque avertissement soulevé par l'outil d'inspection, nous avons l'assurance de livrer un code qui respecte parfaitement les règles de style.

5.2 Mise en place des tests

Pour la réalisation des tests, nous avons créé un dossier puis un fichier par classe à tester. Pour nos tests, nous avons opté pour une approche axée sur les tests unitaires automatisés via le framework *pytest*.

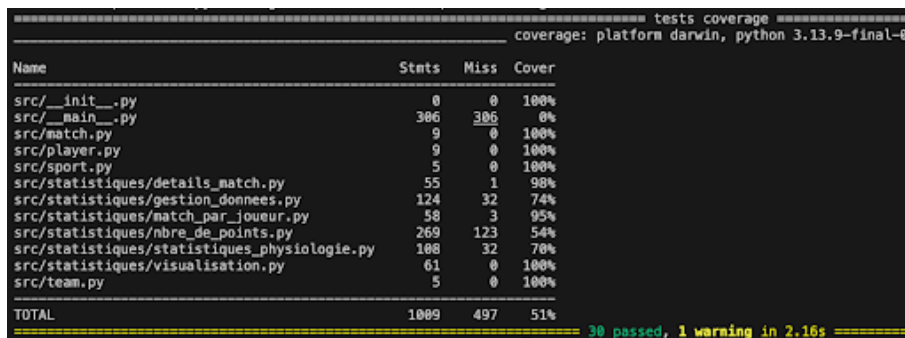
D'autre part, nous avons mesuré notre couverture de code grâce à l'outil *pytest-cov*. Cette métrique, exprimée en pourcentage, nous indique concrètement quelle proportion de notre code source est traversée lors de l'exécution de nos tests unitaires.

Nous avons fait le choix de ne pas tester le fichier *main.py* car c'est un menu d'affichage. Il relie l'utilisateur au reste du programme. Pour vérifier que le menu visuel fonctionne bien et réagit correctement aux choix de l'utilisateur, nous l'avons testé nous-mêmes manuellement.

La majorité de nos classes (*ChampionshipPointsCalculator*, *AnalysePhysiologique*, etc.) dépendent de volumes de données importants. Pour garantir des tests rapides et déterministes, nous avons fait le choix de ne pas charger les vrais fichiers de données lors des tests. Nous avons simulé des objets "matchs" et "équipes" très simples en utilisant *SimpleNamespace*. Cela nous a permis de cibler précisément la logique de nos classes, telle que le ratio de victoire au basket.

De plus, le test de certaines classes nécessite que l'utilisateur saisisse des informations dans le terminal. Nous avons donc utilisé l'outil *unittest.mock.patch*. Cela nous a permis de simuler les réponses d'un utilisateur humain et de vérifier que notre programme réagit correctement face aux erreurs de saisie.

Pour les modules de statistiques générant des graphiques, les tests sont concentrés sur l'exécution sans erreur de la bibliothèque mathématique et sur l'assertion de la présence du fichier final. C'est-à-dire vérifier si le fichier image a bien été créé et sauvegardé sur l'ordinateur. Si le fichier final existe, cela nous prouve que tout le processus de génération a fonctionné correctement.



Name	Stmts	Miss	Cover
src/__init__.py	0	0	100%
src/main.py	306	306	0%
src/match.py	9	0	100%
src/player.py	9	0	100%
src/sport.py	5	0	100%
src/statistiques/details_match.py	55	1	98%
src/statistiques/gestion_donnees.py	124	32	74%
src/statistiques/match_par_joueur.py	58	3	95%
src/statistiques/nbre_de_points.py	269	123	54%
src/statistiques/statistiques_physiologie.py	108	32	70%
src/statistiques/visualisation.py	61	0	100%
src/team.py	5	0	100%
TOTAL	1089	497	51%

30 passed, 1 warning in 2.16s

FIGURE 5.2 : Résultat du test coverage pytest-cov

6 Présentation des résultats

Le développement de notre architecture orientée objet, couplé à l'utilisation des bibliothèques Pandas et Matplotlib, a permis d'aboutir à une application fonctionnelle et dynamique. L'interface reflète directement la modularité de notre code sous-jacent. Cette section présente visuellement les fonctionnalités implémentées, ce qui démontre ainsi la conformité de notre rendu avec le cahier des charges initial.

6.1 L'interface utilisateur

L'application repose sur une interface en ligne de commande structurée autour d'une boucle d'exécution continue. À l'ouverture, l'utilisateur est invité à sélectionner le sport qu'il souhaite analyser parmi les cinq disciplines chargées en mémoire. L'application intègre des mécanismes de sécurité pour fluidifier l'expérience : si un jeu de données est manquant ou corrompu pour un sport précis, le programme ne s'interrompt pas. Il informe l'utilisateur de l'indisponibilité des données et l'invite à choisir une autre option. Une fois le sport sélectionné, un menu d'actions contextuel s'affiche, offrant une arborescence claire entre les requêtes statistiques simples, les visualisations graphiques avancées et le module d'administration des données.

```
==== MENU PRINCIPAL : SPORTS ====
=====
1. Football
2. Tennis
3. Volley
4. Basketball
5. Lol
0. Quitter le programme

Entrez le numéro du sport (ou 0 pour quitter) : 1

==== Actions pour Football ====
1. Consulter les détails d'un match précis
2. Consulter les statistiques d'une équipe
3. Consulter les statistiques d'un joueur
4. Comparateur graphique
5. Statistique physiologique
6. Importer des données (Nouvelle compétition par CSV)
7. Éditer une donnée existante manuellement
8. Retourner au choix des sports

Votre choix (1 à 8) ? (0 pour quitter) : 1

--- ÉTAPE 1 : CHOIX DE LA SAISON ---
1. 2008/2009
2. 2009/2010
3. 2010/2011
4. 2011/2012
5. 2012/2013
6. 2013/2014
7. 2014/2015
8. 2015/2016

Choisissez le numéro de la saison (ou Entrée pour toutes) : 1

--- ÉTAPE 2 : CHOIX DU DUEL ---
Entrez le nom de la première équipe : Arsenal
Entrez le nom de la deuxième équipe : Liverpool

✅ 2 match(s) trouvé(s) :

===== DÉTAILS DU MATCH 1820 =====
🏆 Sport : Football
📅 Saison : 2008/2009
📊 Score : Arsenal 1 - 1 Liverpool

👤 ARSENAL :
> Manuel Almunia, Bacary Sagna, William Gallas, Johan Djourou, Gael Clichy, Denilson, Cesc Fabregas, Alex Song, Samir Nasri, Emmanuel Adebayor, Robin van Persie

+ LIVERPOOL :
> Pepe Reina, Alvaro Arbeloa, Jamie Carragher, Daniel Agger, Emiliano Insua, Xabi Alonso, Lucas Leiva, Dirk Kuyt, Steven Gerrard, Albert Riera, Robbie Keane
```

FIGURE 6.1 : Menu principal du terminal

6.2 Consultation des statistiques

La première fonctionnalité majeure de l'application est sa capacité d'interrogation ciblée. En saisissant le nom d'une entité (un joueur ou une équipe), le programme utilise ses algorithmes sur l'historique des matchs. Afin d'offrir une précision maximale, l'utilisateur peut filtrer sa requête sur une saison spécifique ou demander un bilan global sur l'ensemble de la carrière. L'application s'adapte dynamiquement aux règles inhérentes à chaque discipline sportive : elle calcule et affiche le nombre de matchs nuls si la requête concerne le football, mais occulte automatiquement cette

caractéristique pour des sports comme le tennis ou League of Legends, calculant ainsi un taux de victoire.

```
Appuyez sur Entrée pour continuer...

=====
=== Actions pour Football ===
1. Consulter les détails d'un match précis
2. Consulter les statistiques d'une équipe
3. Consulter les statistiques d'un joueur
4. Compareur graphique
5. Statistique physiologique
6. Importer des données (Nouvelle compétition par CSV)
7. Éditer une donnée existante manuellement
8. Retourner au choix des sports

Votre choix (1 à 8) ? (0 pour quitter) : 2

Entrez le nom de l'équipe (ex: France, Lakers...) : Arsenal

Saisons/Tournois disponibles :
1. 2008/2009
2. 2009/2010
3. 2010/2011
4. 2011/2012
5. 2012/2013
6. 2013/2014
7. 2014/2015
8. 2015/2016
0. Toutes les saisons

Choisissez le numéro de la saison : 8

=== BILAN : Arsenal ===
Equipe : Arsenal
Saison : 2015/2016
Matches joués : 38
Points : 71
Victoires : 20
Victoires domicile : 12
Victoires extérieur : 8
Nuls : 11
Défaites : 7
Défaites domicile : 3
Défaites extérieur : 4
Buts marqués : 65
Buts encaissés : 36
Différence buts : 29
```

FIGURE 6.2 : Résultats des statistiques sur une équipe

6.3 Les comparateurs graphiques

Pour dépasser les limites de la simple lecture de données textuelles, l'application intègre un outil d'analyse comparative visuelle. Ce comparateur permet de mettre en opposition directe deux joueurs ou deux équipes. Une fois les cibles et la temporalité (saison spécifique ou historique complet) sélectionnées par l'utilisateur, l'application sollicite Matplotlib pour générer des diagrammes en barres. Ces graphiques permettent de confronter instantanément les volumes de matchs joués et les taux de victoire.

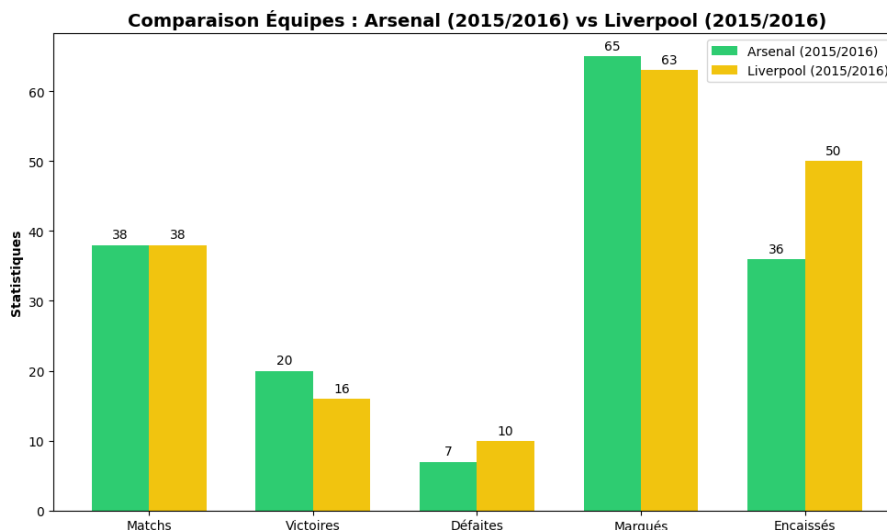


FIGURE 6.3 : Comparaison des résultats de deux équipes

6.4 Analyses physiologiques et data visualisation

Notre classe *AnalysePhysiologique* propose des visualisations croisées. Un premier niveau de lecture prend la forme d'un histogramme des tailles, utile pour identifier le profil physique médian d'un sport donné.

Par ailleurs, nous avons également une méthode pour analyser la corrélation entre les caractéristiques physiques (taille) et les performances (pourcentage de victoire). Pour des bases de données volumineuses contenant des milliers de joueurs, un nuage de points classique aurait été illisible à cause de la superposition des données. Nous avons donc opté pour une carte de chaleur en nid d'abeille (fonction *hexbin*). Cette méthode regroupe les joueurs par alvéoles et utilise une échelle colorimétrique pour mettre en évidence les zones de chaleur physiologique où se concentrent les joueurs les plus victorieux.

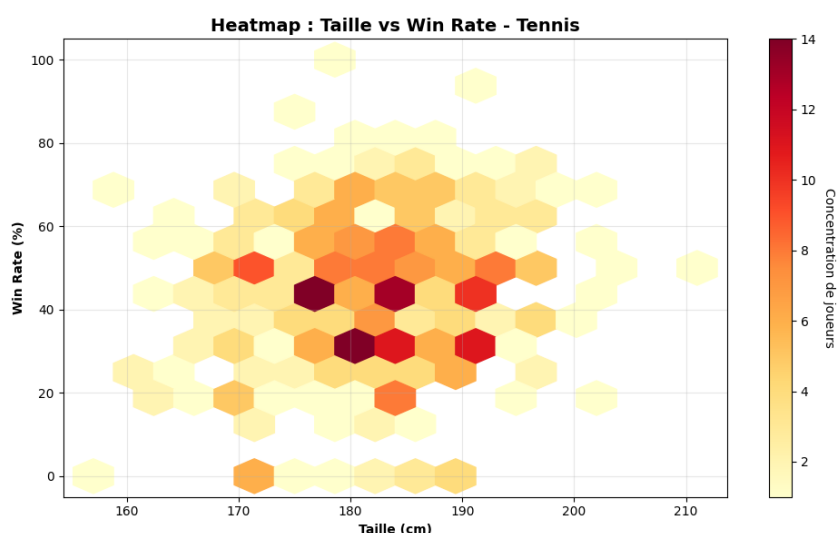


FIGURE 6.4 : Heatmap entre la taille et le pourcentage de victoire

6.5 Édition et mise à jour dynamique

L'application se distingue par sa dimension interactive et pérenne grâce à son module de gestion des données intégré. Elle permet non seulement d'importer de nouveaux fichiers de

compétition, mais aussi d'éditer les données existantes pour corriger d'éventuelles anomalies.

Lorsqu'un utilisateur recherche une entité à modifier, l'algorithme utilise Pandas pour fusionner virtuellement les colonnes. Une fois la cellule ciblée et la nouvelle valeur saisie, la méthode `to_csv()` réécrit physiquement le fichier sur le disque dur, garantissant la persistance de l'information. Immédiatement après la sauvegarde, l'application force les Loaders à rafraîchir les instances en mémoire vive. Ainsi, les modifications manuelles et les nouveaux matchs sont instantanément pris en compte dans les calculs et les graphiques suivants, sans que l'utilisateur n'ait besoin de redémarrer le programme.

```
===== Actions pour Football =====
1. Consulter les détails d'un match précis
2. Consulter les statistiques d'une équipe
3. Consulter les statistiques d'un joueur
4. Comparateur graphique
5. Statistique physiologique
6. Importer des données (Nouvelle compétition par CSV)
7. Éditer une donnée existante manuellement
8. Retourner au choix des sports

Votre choix (1 à 8) ? (0 pour quitter) : 7

--- ÉDITEUR DE DONNÉES MANUEL ---
Que souhaitez-vous modifier ?
1. Un joueur / Une joueuse
2. Une équipe
Votre choix (1 ou 2) : 1

📁 Fichiers de données détectés pour Football :
📄 data/football_european_leagues_tdd/player.csv

Entrez le chemin exact du fichier CSV à modifier (ou Entrée pour annuler) : data/football_european_leagues_tdd/player.csv

Entrez le nom (ou une partie du nom) du joueur à modifier : Lionel Messi

===== Joueur(s) trouvé(s) =====
   id  player_api_id  player_name  birthday  weight (kg)  height (cm)
6169  6176         30901  Lionel Messi  1987-06-24         72.1         170

Entrez le numéro de la ligne à modifier (le nombre tout à gauche) : 6169

Colonnes disponibles : id, player_api_id, player_name, birthday, weight (kg), height (cm)
Entrez le nom exact de la colonne à modifier : height (cm)
Entrez la nouvelle valeur pour 'height (cm)' : 200
```

FIGURE 6.5 : Mise à jour des attributs d'un joueur

7 Conclusion

Ce projet avait pour objectif de développer une application capable de traiter, analyser et comparer des données sportives issues de plusieurs disciplines. L'adoption de la programmation orientée objet a facilité la réponse au cahier des charges en permettant de standardiser des jeux de données. Les fonctionnalités principales ont pu être implémentées : le nettoyage des données brutes avec Pandas, l'analyse physiologique via les cartes de chaleur de Matplotlib, et un module permettant la modification et la sauvegarde des informations en mémoire.

Bien que l'architecture logicielle mise en place soit fonctionnelle, l'outil présente des axes d'amélioration. L'interface reposant exclusivement sur la ligne de commande, le développement d'une interface graphique rendrait l'utilisation générale plus intuitive. De plus, l'actualisation des résultats reposant sur l'importation manuelle de fichiers CSV, une automatisation constituerait une évolution pertinente. En définitive, ce projet nous a amenés à dépasser l'écriture de scripts classiques pour mettre en pratique des concepts d'ingénierie logicielle, tels que l'encapsulation et la séparation des responsabilités.

8 Annexes

8.1 Avis personnels

8.1.1 Jules

Durant ce projet, j'ai pu expérimenter, dans le prolongement du DM donné en IPOO, la réflexion sur la façon d'organiser son code en modules afin de faciliter la lisibilité et le travail d'équipe. Contrairement au DM où l'on nous fournissait le diagramme de classes, j'ai pu comprendre son importance dans un projet : à la fois en amont du développement, pour avoir une direction claire, mais aussi en aval, puisque certaines erreurs de conception peuvent être mises en lumière par le diagramme.

8.1.2 Ilan

8.1.3 Thomas

Dans un premier temps, le travail en équipe nous a appris à nous coordonner efficacement, notamment lors des séances en autonomie. J'ai pour la première fois eu une utilisation collaborative de GitHub qui m'a permis d'apprendre à gérer un code commun sans conflits.

Techniquement, cela a été l'occasion d'appliquer la programmation objet de notre dernier DM sur un cas très concret. De plus, j'ai découvert des outils de test plus variés pour répondre aux caractéristiques de nos différentes classes. Il a été très satisfaisant de passer d'un tableau de données brutes à des statistiques interactives.

8.1.4 Alexandre

Ce projet a été une expérience très formatrice, même s'il a demandé un gros effort d'organisation au départ lorsque nous n'avions pas les bases de données. Le fait de travailler avec des données réelles, avec leurs erreurs et leurs différences d'un sport à l'autre, m'a vraiment fait comprendre l'intérêt d'avoir un code bien structuré. Ce travail m'a surtout permis de comprendre l'intérêt de la Programmation Orientée Objet. Au lieu de coder de façon classique, j'ai appris à bien séparer les tâches avec des classes. Enfin, l'utilisation de Pandas et Matplotlib m'a donné beaucoup de pratique sur le nettoyage et la visualisation de données. En résumé, c'était un excellent pont entre la théorie vue en cours et la réalité du développement d'une application.